Rapport de Projet: Architecture Lambda appliquée à un flux Twitch

Nada Aleian – Eva Lansalot

1. Introduction

Dans le cadre de ce projet, l'objectif est d'analyser un chat Twitch en direct afin de cartographier son activité et d'évaluer le niveau de toxicité des messages échangés.

Pour répondre aux défis imposés par la vélocité et le volume de ces données, nous avons déployé une stack Big Data complète orchestrée sous Docker. Le socle repose sur Apache Kafka pour l'ingestion des flux, Apache Spark pour le traitement distribué, Cassandra comme base de données NoSQL pour le Serving Layer, et Grafana pour la restitution visuelle.

Le système s'articule autour d'une Architecture Lambda, combinant un traitement en temps réel dans le Speed Layer avec Spark Streaming et un traitement d'historisation profond dans le Batch Layer avec Spark.

2. Contexte et données choisies

2.1 Contexte et limites inhérentes

Les chats de streaming génèrent des rafales de messages très denses, particulièrement lors de "pics de hype". Le langage y est hautement non structuré : argot, émojis, fautes de frappe et spam intentionnel.

Note sur la plateforme : Bien que ce projet cible Twitch, l'API de cette plateforme intègre déjà une modération native stricte (AutoMod). Pour une analyse de toxicité plus probante et brute, une plateforme moins régulée comme Kick constituerait à l'avenir un cas d'étude générant des résultats potentiellement plus significatifs.

2.2 Données utilisées

Les données sont captées par un script Python (Producer) connecté à l'interface IRC de Twitch. Le protocole IRC brut ne fournissant que l'utilisateur et le texte, notre Consumer enrichit chaque événement en y injectant un Timestamp système avant de l'envoyer dans Kafka.

Chaque message injecté comprend donc :

- user (Le viewer)
- message (Contenu du chat)

- timestamp (Timestamp du message)

3. L'Architecture développée

Le système est découpé en rôles stricts pour garantir la scalabilité :

1. Ingestion : Le Producer Python intercepte le chat et publie dans un topic Kafka.
2. Speed Layer : Spark Streaming s'abonne au topic Kafka, effectue des agrégations temporelles (micro-batching), et met à jour les résultats directement dans Cassandra.
3. Data Lake & Batch Layer : Un script Consumer Kafka sauvegarde les données brutes dans un fichier JSON. Spark Batch lit ce JSON, le convertit en Parquet, réalise des calculs lourds (surtout parce qu'il y a un grand nombre de données), et pousse les agrégats finaux dans Cassandra.
4. Serving Layer : Cassandra stocke les tables pré-calculées (Batch) et les tables en mouvement (Streaming), qui sont ensuite requêtées par Grafana.

NB: Grafana a été mise en place à la suite de la présentation en classe du projet.

4. Les Traitements appliqués

4.1 Traitement Spark Batch et le choix du format Parquet

Le job Batch a pour rôle d'analyser l'historique complet d'une session de stream. Le format JSON est lourd et coûteux à parser pour un moteur distribué. En convertissant les données brutes en Parquet, un format de stockage en colonne, nous avons drastiquement optimisé les performances. En effet, le volume de données stocké sur le disque est réduit et Spark ne scanne que les colonnes nécessaires, accélérant massivement les requêtes d'agrégation.

Traitements effectués : Comptage total, moyenne de messages par utilisateur, Top 10 des utilisateurs, et extraction des mots les plus fréquents.

4.2 Traitement Spark Streaming (Speed Layer)

Spark Streaming lit Kafka en continu. Puisque Spark repose techniquement sur un moteur de micro-batching (et non du streaming événementiel pur), nous avons défini des fenêtres glissantes de 10 secondes.

Traitements effectués : Comptage de l'activité globale par fenêtre temporelle, classement des utilisateurs actifs à l'instant T.

4.3 Analyse de la toxicité

Un traitement parallèle (applicable en Batch et Streaming) filtre les messages confrontant le texte à une liste noire (scam, idiot, trash, etc.).

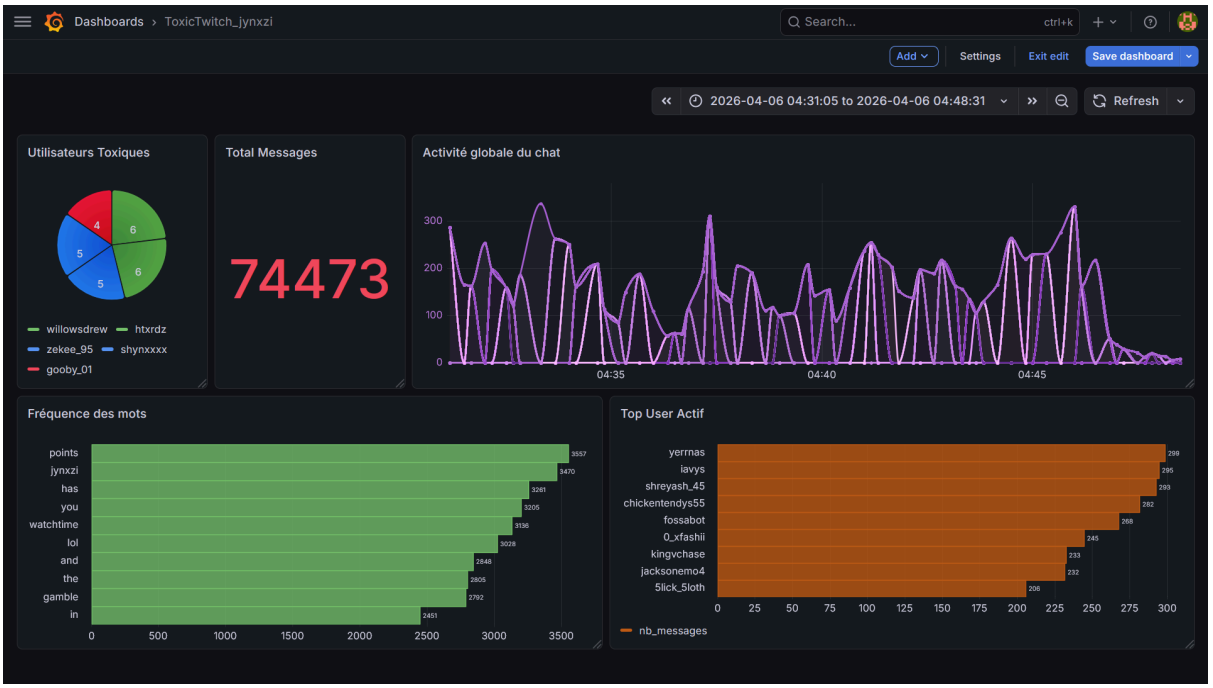
Traitements effectués : Identification des utilisateurs ayant le plus fort taux d'infraction et calcul du volume de toxicité par fenêtre de temps.

5. Résultats obtenus et Observations

Contrairement à de simples tests à vide, l'exécution sur un flux réel a mis en évidence plusieurs comportements mesurables :

5.1 Résultats de l'analyse

- Les résultats Batch montrent systématiquement que les "Top Utilisateurs" sont des bots de modération ou d'animation (ex: *streamelements* ou *fossabot* avec des milliers de messages), loin devant les utilisateurs humains.
- Le Top Mots est dominé par des émojis textuels spécifiques à la culture Twitch (ex: *KEKW*, *LUL*, *Pog*), confirmant la nature informelle des données.
- Les utilisateurs identifiés comme "toxiques" par notre script présentaient souvent des occurrences multiples (spam d'insultes), validant le fonctionnement de la détection, bien que des faux positifs existent si on sort les mots de leurs contextes. Cependant la liste des mots toxiques a dû être modifiée car la modération active pendant les lives ne nous permettait pas d'obtenir les chats réellement toxiques.



5.2 Comportement de l'infrastructure

Notre environnement matériel (PC avec 8 à 16 Go de RAM) a dicté la cadence du projet.

Les logs de Spark Streaming affichaient régulièrement des avertissements (“Current batch is falling behind”) car le traitement d'un micro-batch de 5 secondes prenait parfois jusqu'à 40 secondes à cause du manque de cœurs CPU alloués.

Face à ces lenteurs, nous avons fait le choix stratégique de ne lancer le job Batch qu'une seule fois, à la fin de la diffusion du stream, plutôt que de le planifier à intervalles réguliers car ça aurait fait crasher les machines par manque de mémoire.

6. Analyse critique et Améliorations possibles

6.1 Limites des traitements analytiques

La détection par liste de mots est rigide. L'intégration d'un modèle de Machine Learning (NLP) ou l'appel à une API d'analyse de sentiment permettrait de comprendre le contexte et de réduire les faux positifs.

6.2 Améliorations de l'Architecture

Pour passer ce projet à une échelle de production industrielle, l'architecture elle-même devrait évoluer.

Tout d’abord, Spark Streaming gère des "mini-lots". Pour une réactivité à la milliseconde (vrai temps réel), migrer la Speed Layer vers Apache Flink ou Kafka Streams serait technologiquement plus pertinent et moins gourmand en mémoire vive.

Aussi, une configuration plus fine de Spark est plus intéressante pour répartir correctement la charge sur les threads du processeur. C’est ce qui n'a pas pu être totalement exploité sur nos machines personnelles.

7. Conclusion

Ce projet démontre avec succès l'implémentation de bout en bout d'une architecture Lambda pour le traitement Big Data. La combinaison Kafka, Spark et Cassandra s'est avérée fonctionnelle pour ingérer, transformer et stocker des flux de données non structurés.

Si les résultats extraits offrent déjà des insights clairs sur l'activité d'une communauté (pics d'audience, typologie des utilisateurs, modération basique), la réalité matérielle a mis en exergue l'importance vitale du dimensionnement des clusters. Ce travail constitue une base solide pour de futures itérations orientées vers le vrai temps réel et l'intelligence artificielle (NLP).